Classification of News Articles Using Machine Learning Models

Hamza Malik
26100060

Maaz Kashif
26100182

Faaiz Umer
26100088

Hamza Bin Mubeen
26100353

Ahsan Usman
26100009

ABSTRACT

This report describes the implementation of a classification model for news articles, leveraging web scraping to create datasets and using various machine learning models for classification. The categories chosen for classification were Sports, Entertainment, Business, World, and Science/Technology. Each step of the process is outlined in detail, from data collection to model evaluation.

1 INTRODUCTION

We were tasked with classifying various articles from the internet into five categories: Sports, Entertainment, Business, World, and Science/Technology. This document outlines the methodologies employed, including web scraping, data preprocessing, and three machine learning models—Multinomial Naive Bayes, Neural Networks, and K-Nearest Neighbors (KNN). Results and recommendations for future improvements are also discussed.

2 METHODOLOGY

2.1 Data Collection

The dataset was created by extracting and processing articles from multiple Urdu news websites: **Geo**, **Jang**, **Express**, **Samaa**, **Dunya**, and **Dawn**. Customized scraping scripts were implemented to accommodate the unique HTML structures of these websites. Some examples are:

- **Express**: Article content was extracted from: `<span class='story-text'>`.
- **Geo**: Content was retrieved from: `<div class='content-area'>`.
- **Jang**: Articles were located in: `<div class='news-detail'>`.

The scraping process began by visiting the recommended websites and collecting their HTML links along with pseudo labels, such as `/Sports` or similar indicators of the article category. These pseudo labels were standardized into gold labels like *Entertainment* or *Business*.

The scripts accessed the columns (e.g., *Sports*, *World*) on each website, inspected their HTML structure, and extracted article titles along with links to the full articles. These titles and links were stored in a data frame. Each article was then accessed using its link, and the content was scraped into the `Content` field of the data frame. Pre-existing column labels were standardized as gold labels.

The final dataset included the following fields:

- **ID**: A unique identifier for each article (automatically assigned).
- **Title**: The headline of the article.
- **Content**: The main body text scraped from the article.
- **Link**: The URL of the article.
- **Gold Label**: The standardized classification label.

The column labels on some websites were mapped to standardized categories. For example:

- `saqafat` on *Express* was categorized as *Entertainment*.
- `science` was mapped to *Science/Technology*.
- `health-science` on *Jang* was also mapped to *Science/Technology*.
- `International` was categorized as *World*.

2.2 Processing

To ensure diversity and avoid overfitting, the scraping was performed incrementally at timely intervals of one week across four weeks. This approach captured updates in the news and created a more representative and dynamic dataset.

A separate script was developed to preprocess the article content by removing stop words. This reduced the dataset's dimensionality, removed noise, and enabled models to learn more effectively from the informative content.

Error-handling mechanisms were also included in the scripts to manage missing or inaccessible articles to ensure the process continued without disruption.

Some challenges that we kept in mind during this process:

(1) **HTML Structure Dependency**: Changes in website structures could require updates to the scripts.
(2) **Missing Content**: Some articles might be unavailable or inaccessible and be excluded to maintain dataset quality.

3 IMPLEMENTATION

3.1 Multinomial Naive Bayes

The first model, Multinomial Naive Bayes (MNB), was chosen for its simplicity, efficiency, and strong performance with text data. MNB assumes feature independence, a simplifying assumption that helps it efficiently handle high-dimensional data, such as text with numerous features (words). It is particularly suited for tasks with discrete features like word counts.

3.1.1 Data Preprocessing.

- **Cleaning Stop Words**: The dataset was cleaned to remove stop words using a custom Urdu stop word list and a custom Python script. This step eliminated words that do not provide meaningful information for classification, reducing noise and improving model accuracy.
- **Removing Punctuation**: Punctuation marks were removed from the text to ensure only relevant words were used as features, minimizing noise in the data.

3.1.2 Feature Representation. The **Bag-of-Words (BoW)** approach was used to vectorize the text for the Multinomial Naive Bayes model. This method represents text by counting the frequency of

each word in the corpus, aligning well with the Naive Bayes assumption that word occurrences follow a multinomial distribution.

- **Why BoW?** BoW was chosen over alternatives like One-Hot encoding because it captures both the presence and frequency of words, providing richer feature information about importance.
- **Training and Testing Consistency**: The BoW model was fitted on the training data to ensure proper learning of word frequencies, allowing both training and test datasets to use the same vocabulary and preventing zero probabilities for words in the test set not seen in the training set.

3.1.3 Model Training and Testing.

- **Laplace Smoothing**: Laplace smoothing was applied during training to prevent zero probabilities, ensuring every word had a non-zero probability, even if absent from the training data for a specific class.
- **Model Training**: The Naive Bayes classifier was trained on vectorized data, computing class and feature probabilities, and then used to predict the class for each test document.

3.2 Neural Network

The second model, a neural network, was chosen for its ability to handle the complexity of the scraped dataset, which has varied content and structure. The network captures intricate patterns in text that simpler models might miss, particularly with the high-dimensional Bag-of-Words (BoW) representation. By using multiple layers, the network learns both basic and complex relationships, enhancing classification performance. Its flexibility allows for experimentation with architectures to optimize accuracy.

3.2.1 Data Preprocessing and Feature Engineering.

- **Cleaning Stop Words**: The dataset is first cleaned of any stop words using the same method as discussed earlier, to improve the model's training performance and accuracy.
- **Removing Irrelevant Data**: The ID and Link columns are dropped. The ID column is removed to reduce dimensionality as it contributes no information, and the Link column is dropped to prevent overfitting by linking classes to specific sources.
- **Feature Representation**: The remaining columns, "Title" and "Content", are represented using the **Bag-of-Words (BoW)** approach. It is applied separately to both "Title" and "Content" to capture their respective feature-specific patterns.

3.2.2 Neural Network Architecture. The neural network is designed with the following architecture:

- **Input Layer**: Dimension of 16,973, corresponding to the number of unique words.
- **Hidden Layers**: Two hidden layers with 1024 and 512 units were used respectively. This architecture was chosen after experimentation, as adding more layers reduced training error but caused overfitting, making the model perform poorly on unseen data. A two-layer setup balances complexity and generalization.
- **Output Layer**: A layer with 5 dimensions, one for each class.
- **LeakyReLU Activation**: Used after each hidden layer to introduce non-linearity and prevent dead neurons. It allows a small gradient for negative inputs, mitigating the vanishing gradient problem faced when using functions like sigmoid or tanh.
- **Adam Optimizer**: Chosen for its ability to adapt learning rates for each parameter. Combining the benefits of momentum (like in SGD) with adaptive learning rates, it proved well-suited for efficiently training the complex neural network.
- **Loss Function**: Cross-Entropy Loss was used for multi-class classification, as it measures the difference between the predicted and true probability distributions, aligning with the goal of classifying articles into multiple categories.

3.2.3 Training Process and Generalization.

- **Mini-Batch Training**: Instead of using the entire dataset for each iteration, mini-batches were used to speed up training and improve generalization. This approach accelerates learning and reduces the risk of overfitting.

3.3 K-Nearest Neighbors (KNN)

The third model, K-Nearest Neighbors (KNN), was chosen for its simplicity and ability to classify instances based on similarity. KNN assumes similar instances are close in feature space, making it suitable for text classification, where high-dimensional vectors can be compared using distance measures like Euclidean distance.

3.3.1 Data Preprocessing.

- **Cleaning Stop Words**: Stop words were removed to reduce noise in the feature space, as KNN relies on distance metrics, and irrelevant words could distort the similarity between data points.
- **Removing Punctuation**: Punctuation marks were removed to prevent them from being treated as features, which could add unnecessary complexity and lower accuracy.

3.3.2 Feature Representation. We once again chose the BoW approach specifically for KNN because it enabled accurate distance calculations between documents with similar word frequencies, resulting in smaller distances in the feature space and better classification.

3.3.3 Model Training and Testing.

- **Distance Calculation**: In this implementation, the **Euclidean distance** metric was used to measure the similarity between the feature vectors.
- **K Value**: K = 4 was chosen after experimenting with different values, striking a balance between overfitting and underfitting. A smaller k (e.g., k=1) made the model sensitive to noise, while a larger k (e.g., k=20) made the model too general, missing important patterns and information in the data.
- **Prediction**: After training the model on the vectorized training data, predictions were made on the test data based

on the distance of each test instance from the training instances.

4 FINDINGS

Table 1: Accuracy Performance Metrics

Model	Accuracy
Multinomial Naive Bayes	0.9643
Neural Network	0.9732
KNN Classifier	0.7857

Table 2: Precision Performance Metrics

Model	Precision
Multinomial Naive Bayes	0.9653
Neural Network	0.9732
KNN Classifier	0.8064

Table 3: Recall Performance Metrics

Model	Recall
Multinomial Naive Bayes	0.9643
Neural Network	0.9733
KNN Classifier	0.7857

Table 4: F1-Score Performance Metrics

Model	F1-Score
Multinomial Naive Bayes	0.9642
Neural Network	0.9730
KNN Classifier	0.7889

4.1 Multinomial Naive Bayes (MNB)

MNB performed well for this classification task, but it did not match the performance of the neural network for several reasons:

- **Preprocessing**: While removing stop words, punctuation, and irrelevant columns reduced noise and improved performance, MNB could not fully utilize the deeper semantic relationships within the text compared to the neural network.
- **Naive Bayes Assumptions**: The independence assumption, while not a significant limitation for MNB in this task, inherently restricted its ability to capture relationships between words. The neural network, with its ability to model dependencies and interactions, outperformed MNB in capturing the complex structures of the data.

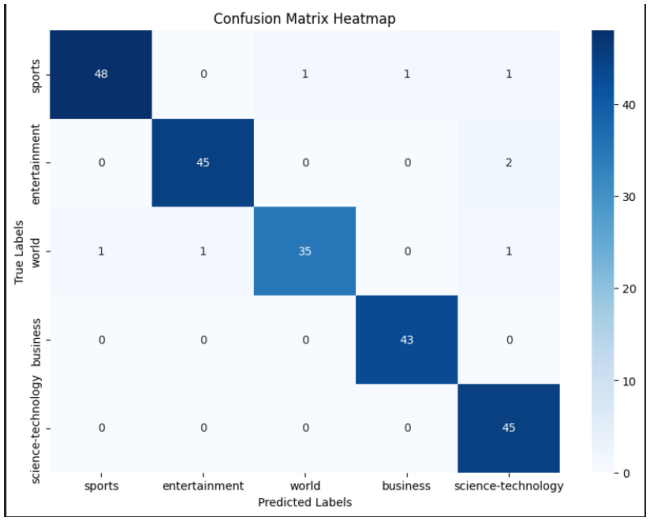


Figure 1: Multinomial Naive Bayes Heat Map

4.2 Neural Networks

The Neural Network model achieved the best performance, with several factors contributing to its success:

- **Model Complexity**: The neural network outperforms other models due to its ability to model complex, non-linear relationships through its layered design. By employing multiple hidden layers, the network creates deep representations of the input data. Activation functions like LeakyReLU introduce non-linearity, allowing each neuron to capture non-linear transformations of inputs. This enables the network to learn intricate patterns and feature connections that linear models or those assuming feature independence cannot.
- **Optimization Strategy**: The **Adam optimizer** and **mini-batch training** were essential in speeding up the training process, ensuring faster convergence, and improving the model's ability to generalize to unseen data. This efficient learning process contributed to the model's high performance, particularly in terms of accuracy and precision.

Together, these factors allowed the Neural Network to outperform other models in terms of accuracy, achieving superior performance across all metrics, including precision, recall, and F1 scores.

4.3 K-Nearest Neighbors (KNN)

While KNN is a simple and intuitive model, it faced significant challenges when applied to this task:

- **Feature Representation**: KNN relies on distance-based methods, which struggle with sparse, high-dimensional text data like Bag-of-Words. Unlike MNB, which uses probabilistic distributions, or Neural Networks, which model non-linear patterns, KNN has difficulty capturing deeper relationships and generalizing effectively in sparse datasets.
- **Curse of Dimensionality**: The high-dimensional BoW representation worsened the curse of dimensionality for KNN. As the number of features grew, distance differences

between data points diminished, making it harder for KNN to distinguish classes, especially in sparse text data.

- **Scalability**: KNN is computationally expensive for high-dimensional datasets like text, as it calculates distances to all training documents for each test instance. This scalability issue makes it impractical for large datasets, especially compared to the more efficient Naive Bayes and Neural Networks.

[b]

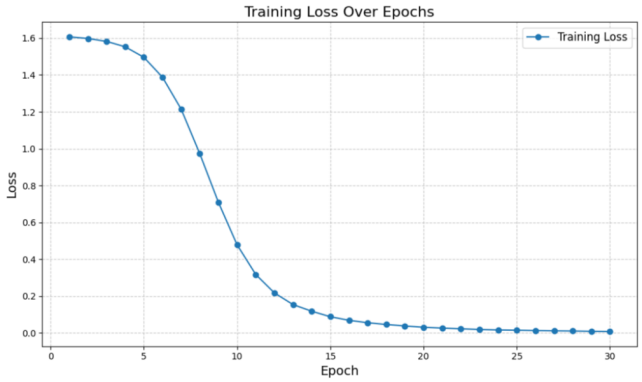


Figure 2: Neural Network Loss Curve

[b]

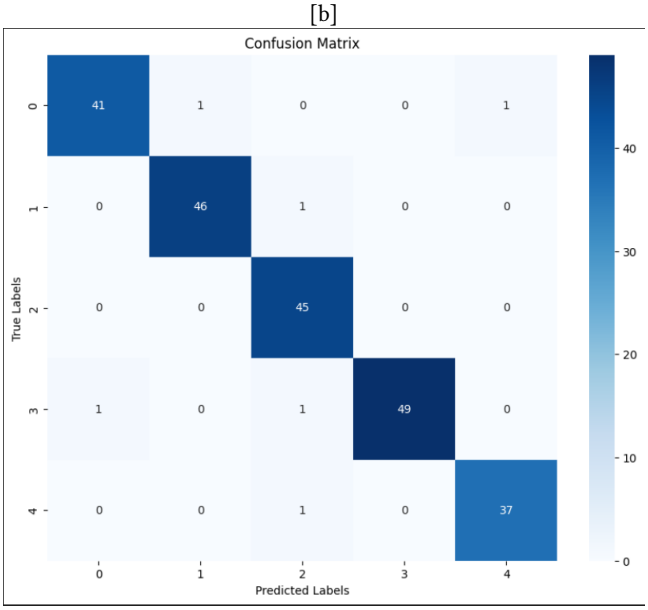


Figure 3: Neural Network Heat Map

In summary, the performance of these models is largely influenced by their ability to handle the sparsity and high-dimensional nature of text data. **MNB** and **Neural Networks** performed well due to their effective preprocessing, feature representation, and model architecture. In contrast, **KNN** struggled with the curse of dimensionality, distance sensitivity, and scalability, which hindered its performance on this task. These insights highlight the importance of selecting the appropriate model based on the characteristics of the data and the specific problem at hand.

5 CONCLUSION

This project highlights the importance of preprocessing and model selection in text classification. While Neural Networks offer superior performance, simpler models like Naive Bayes are viable for lightweight applications. Future work could explore deep learning techniques like transformers for enhanced classification.

INDIVIDUAL CONTRIBUTIONS

- **Hamza Malik**: Developed data scraping scripts and initial pre-processing.
- **Maaz Kashif**: Handled data pre-processing and MNB implementation.
- **Faaiz Umer**: Handled data pre-processing and Neural Network implementation.
- **Hamza Bin Mubeen**: Handled data pre-processing and KNN implementation.
- **Ahsan Usman**: Handled Final Analysis and Documentation.